# Language Translation Tool

CodeAlpha AI Internship — Task 1

| | |
|---|---|
| **Project type** | Artificial Intelligence / Python |
| **Deliverable** | Project report with implementation source code |
| **Prepared for** | CodeAlpha AI Internship Task |

## Implementation note

This document includes a working starter implementation and the steps needed to run and test it. Update the sample content, test it in your own environment, and add your own screenshots or observations before submitting. Do not claim a test result that you have not personally verified.

# 1. Introduction

The Language Translation Tool accepts text, lets the user choose a source and target language, sends the text to a translation service, and displays the translated result. The interface is built with Streamlit and the translation request is handled by the deep-translator package.

The source language can be set to Auto-detect. The sample interface includes English, Telugu, Hindi, Tamil, Kannada, Malayalam, French, German, Spanish, and Japanese.

# 2. Objectives

• Provide a simple text input interface.
• Allow source and target language selection.
• Translate user-entered text through an online translation service.
• Display and download the translated text.
• Handle empty input and network/API errors gracefully.

# 3. System requirements

Python 3.9 or later, an internet connection, and a browser. Install the required packages with **pip install streamlit deep-translator**.

# 4. Working methodology

1. The user selects source and target languages.
2. The user enters text and selects Translate.
3. The application validates the input.
4. GoogleTranslator sends the translation request to the online service.
5. The returned text is shown in the result area and can be downloaded.

# 5. How to run

Save the appendix code as **app.py**. Open a terminal in that folder, install the packages, and run **streamlit run app.py**. Open the local URL printed by Streamlit in a browser. Internet access is required for translation.

# 6. Testing checklist

Try a short English sentence translated to Telugu; try Auto-detect; try an empty input; try selecting the same source and target language; and test the download button. Record the actual results from your own run.

# 7. Limitations and future enhancement

Translation quality can vary by language and context, and the online service may impose availability or usage restrictions. Future improvements include text-to-speech, translation history, more languages, and an official paid translation API with an API key stored securely in environment variables.

# 8. Conclusion

The project demonstrates a basic end-to-end translation workflow with a graphical interface, language selection, online translation, result display, and error handling.

# Appendix: Complete source code

```
"""
Language Translation Tool
Install: pip install streamlit deep-translator
Run: streamlit run app.py
"""
import streamlit as st
from deep_translator import GoogleTranslator

LANGUAGES = {
    "English": "en", "Telugu": "te", "Hindi": "hi", "Tamil": "ta",
    "Kannada": "kn", "Malayalam": "ml", "French": "fr",
    "German": "de", "Spanish": "es", "Japanese": "ja"
}

st.set_page_config(page_title="Language Translation Tool", page_icon="■")
st.title("Language Translation Tool")
st.write("Translate text between supported languages.")

source_name = st.selectbox("Source language", ["Auto-detect"] + list(LANGUAGES))
target_name = st.selectbox("Target language", list(LANGUAGES), index=1)
text = st.text_area("Enter text", height=160, placeholder="Type or paste text here...")

if st.button("Translate", type="primary"):
    if not text.strip():
        st.warning("Please enter some text first.")
    elif source_name == target_name:
        st.info("Choose two different languages.")
    else:
        try:
            source_code = "auto" if source_name == "Auto-detect" else LANGUAGES[source_name]
            result = GoogleTranslator(source=source_code,
                                      target=LANGUAGES[target_name]).translate(text)
            st.subheader("Translated text")
            st.text_area("Result", value=result, height=160)
            st.download_button("Download translation", result,
                               file_name="translation.txt", mime="text/plain")
        except Exception as error:
            st.error("Translation could not be completed. Check your internet connection and try again.")
            st.caption(f"Technical detail: {error}")
```